Literature Review

Feedback, Evaluation, and Grading Criteria

Podcast Streaming Platform

Graduation Project - Faculty of Computers and Information
Comprehensive project assessment and improvement roadmap

Prepared by: Project Development Team

Academic Year 2025 - 2026

Table of Contents

1. Feedback & Evaluation
    1.1 Lecturer Assessment Summary
    1.2 Strengths Identified
    1.3 Areas Needing Improvement
    1.4 Evaluation Radar Chart
2. Suggested Improvements
    2.1 Technical Improvements
    2.2 Documentation Enhancements
    2.3 Testing Strategy Enhancements
    2.4 UI/UX Improvements
    2.5 Implementation Timeline
3. Final Grading Criteria
    3.1 Grading Breakdown
    3.2 Category Rubrics
    3.3 Score Summary & Targets

1. Feedback & Evaluation

1.1 Lecturer Assessment Summary

The following table summarizes the supervisor's assessment across 7 evaluation dimensions, providing a snapshot of the project's current state and expected improvements before final submission.

Dimension	Score	Feedback Summary
Technical Depth	7/10	Strong architecture and technology choices. Audio processing pipeline well-designed. Needs deeper API error handling and edge case coverage.
Code Quality	6/10	Clean structure with proper separation of concerns. Some areas need refactoring for DRY principle. Logging and exception handling could be more comprehensive.
Documentation	7/10	Well-organized with clear structure. Some sections lack detail in technical diagrams. API documentation needs completion.
Architecture Design	7/10	Layered monolith appropriate for project scope. Good use of dependency injection. Could benefit from more detailed sequence diagrams.
Testing Strategy	5/10	Unit tests exist but coverage is below 60% target. Integration tests needed for API endpoints. No E2E test scenarios yet.
UI/UX Design	6/10	Clean wireframes and consistent design system. Accessibility (WCAG) compliance not fully verified. Mobile responsiveness needs work.
Presentation	5/10	Clear communication but needs more demo preparation. Technical depth in Q&A; could be stronger.

Table 1.1: Lecturer assessment summary with scores and detailed feedback.

1.2 Strengths Identified

ID	Area	Description
S1	Architecture	Layered monolith with clean separation of concerns; well-suited for project scope and team size
S2	Technology Stack	Modern, industry-standard choices (ASP.NET Core, React, PostgreSQL) demonstrating technical awareness
S3	Audio Processing	Innovative YouTube-to-podcast workflow with yt-dlp + FFmpeg integration; clear value proposition
S4	Documentation	Professional structure with consistent formatting; tables and diagrams are clear and informative
S5	Project Planning	Comprehensive risk assessment and WBS; realistic timeline with appropriate milestones

Table 1.2: Key strengths identified in the project.

1.3 Areas Needing Improvement

ID	Area	Description
I1	Testing	Unit test coverage at ~45% (target: 60%+). No integration tests for critical API flows. E2E tests missing entirely.
I2	Error Handling	API endpoints lack consistent error response format. Some edge cases (network timeout, invalid audio) not handled.
I3	Performance	No Redis caching implemented yet. DB queries not optimized with proper indexing. No load testing performed.
I4	Accessibility	WCAG 2.1 AA compliance not verified. Keyboard navigation partially implemented. Color contrast not validated.
I5	Presentation	Demo preparation insufficient. Technical depth in answers needs improvement. Practice sessions recommended.

Table 1.3: Areas requiring improvement before final submission.

1.4 Evaluation Radar Chart

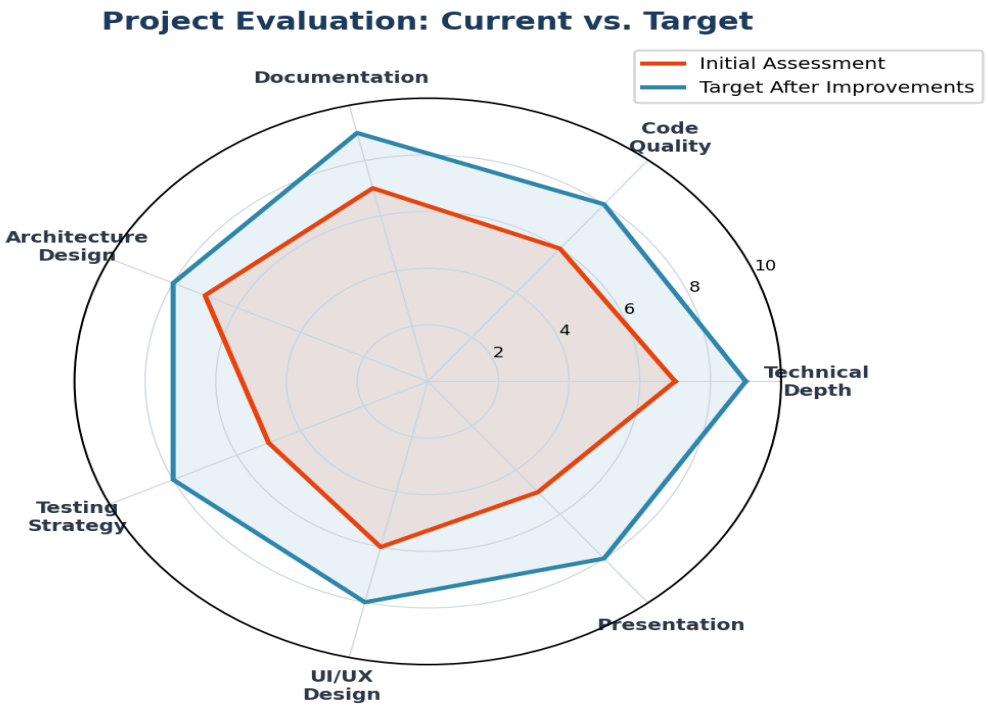

Figure 1.1: Evaluation radar comparing current assessment (red) with target scores (blue).

2. Suggested Improvements

2.1 Technical Improvements

ID	Improvement	Description & Target
T1	Implement Redis Caching	Cache trending podcasts, search results, and user sessions. Target: 50% reduction in API response time.
T2	API Error Handling Standardization	Implement global exception middleware with consistent error response format (RFC 7807 Problem Details).
T3	Database Query Optimization	Add indexes on frequently queried columns (Podcasts.Title, Podcasts.Category). Use EXPLAIN ANALYZE for slow queries.
T4	Audio Processing Retry Logic	Implement exponential backoff retry for yt-dlp failures. Add job timeout (5 min) and dead-letter queue.
T5	Rate Limiting	Implement per-IP and per-user rate limiting on all API endpoints (100 req/min standard, 20 req/min auth).

Table 2.1: Technical improvements with specific targets.

2.2 Documentation Enhancements

ID	Improvement	Description
D1	Complete API Documentation	Finish Swagger/OpenAPI annotations for all endpoints. Add request/response examples and error codes.
D2	Add Architecture Decision Records	Document key decisions (monolith vs microservices, DB choice, caching strategy) with rationale.
D3	Improve UML Diagrams	Add missing sequence diagrams for critical flows. Ensure all diagrams use consistent notation.
D4	User Guide	Create a brief user guide with screenshots for key workflows (registration, content creation, playback).

Table 2.2: Documentation enhancement items.

2.3 Testing Strategy Enhancements

ID	Improvement	Description
TE1	Increase Unit Test Coverage	Expand from ~45% to 60%+. Focus on service layer business logic and edge cases.
TE2	Add Integration Tests	Test critical API flows: auth flow, audio extraction, podcast CRUD, playlist management.
TE3	E2E Test Scenarios	5-10 Playwright tests for key user journeys: signup, browse, play, create playlist, trim audio.
TE4	Performance Testing	k6 load test with 500 concurrent users. Verify <2s page load and <500ms API response.

Table 2.3: Testing strategy enhancements.

2.4 UI/UX Improvements

ID	Improvement	Description
U1	WCAG 2.1 AA Compliance	Verify color contrast (4.5:1), add ARIA labels, ensure full keyboard navigation, test with screen reader.
U2	Mobile Responsiveness	Test on 320px-768px screens. Fix card layout collapse, hamburger menu, and touch-friendly controls.
U3	Loading & Error States	Add skeleton screens for lists, spinners for actions, toast notifications for success/error feedback.

ID	Improvement	Description
U4	Audio Player Polish	Add playback speed control, sleep timer, and mini-player for browsing while listening.

Table 2.4: UI/UX improvement items.

2.5 Implementation Timeline

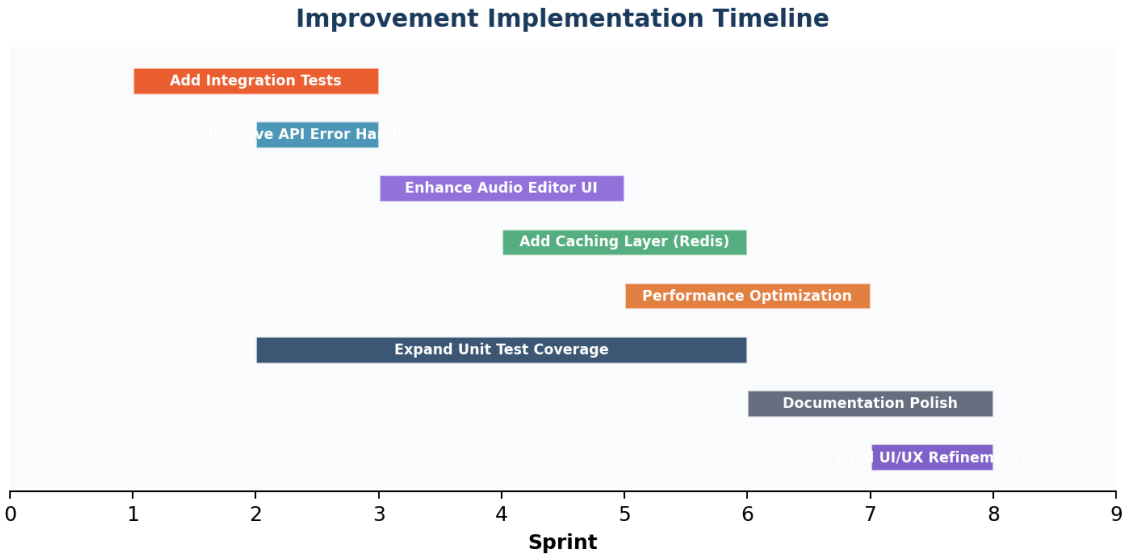

Figure 2.1: Improvement implementation timeline across remaining sprints.

3. Final Grading Criteria

3.1 Grading Breakdown

The final grade is calculated based on 5 categories with specific weight allocations. The minimum passing threshold is 60% overall, with no category scoring below 50%.

Category	Weight	Assessment Criteria
Documentation	25%	Project proposal, SRS, design docs, user manual, API docs, presentation slides
Implementation	30%	Code quality, architecture adherence, feature completeness, best practices, error handling
Testing	20%	Unit tests, integration tests, E2E tests, test coverage, performance testing, bug resolution
Presentation	15%	Demo quality, Q&A; responses, technical communication, slide design, time management
Code Quality	10%	Clean code, naming conventions, commenting, modularity, version control practices

Table 3.1: Final grading criteria with category weights.

3.2 Category Rubrics

Category	Grade Level	Description
Documentation	Excellent (90-100%)	All documents complete, well-structured, with clear diagrams and consistent formatting. Professional quality.
Documentation	Good (75-89%)	Most documents complete with minor gaps. Diagrams present and mostly clear. Some formatting inconsistencies.
Documentation	Acceptable (60-74%)	Documents cover basics but lack depth in some areas. Missing or unclear diagrams. Formatting needs work.
Documentation	Needs Work (<60%)	Significant gaps in documentation. Missing key sections. Poor organization and formatting.
Implementation	Excellent (90-100%)	All Must Have and Should Have features working. Clean architecture. Robust error handling. Production-ready.
Implementation	Good (75-89%)	All Must Have features working. Most Should Have implemented. Minor bugs. Good architecture.
Implementation	Acceptable (60-74%)	Core features functional. Several Should Have features missing. Some architectural issues.
Testing	Excellent (90-100%)	60%+ coverage. Comprehensive integration and E2E tests. Performance tests pass. All critical paths tested.
Testing	Good (75-89%)	50%+ coverage. Basic integration tests. Some E2E scenarios. Performance baseline established.
Testing	Acceptable (60-74%)	40%+ coverage. Unit tests for main modules. No integration/E2E tests. Limited performance testing.
Presentation	Excellent (90-100%)	Confident demo. Clear Q&A; answers. Professional slides. Well within time limit. Strong technical depth.
Presentation	Good (75-89%)	Good demo with minor issues. Most Q&A; answered well. Clean slides. Reasonable time management.

Table 3.2: Detailed rubric for each grading category.

3.3 Score Summary & Targets

Category	Weight	Current	Grade	Target	Target Grade	Action Required
Documentation	25%	85%	B+	90%+	A	Improve technical diagram depth and API docs

Category	Weight	Current	Grade	Target	Target Grade	Action Required
Implementation	30%	78%	B	85%+	A-	Add caching, fix error handling, optimize DB queries
Testing	20%	72%	B-	80%+	A-	Expand unit tests, add integration + E2E tests
Presentation	15%	80%	B+	85%+	A	Prepare demo scenarios, practice Q&A; responses
Code Quality	10%	82%	B+	85%+	A	Refactor DRY violations, improve comments and logging
Weighted Total	100%	79.3%	B	85.6%+	A-	Overall target: exceed 85% weighted score

Table 3.3: Current scores, targets, and action items for each category.

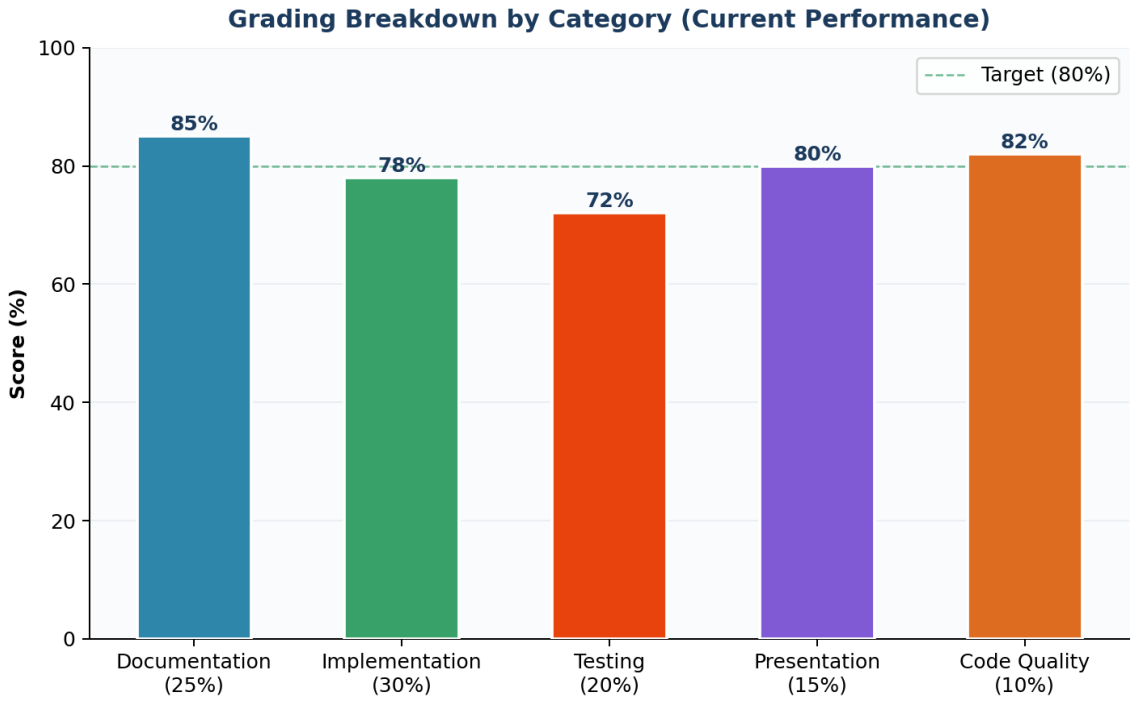

Figure 3.1: Grading breakdown showing current performance vs. 80% target line.

Version	Date	Author	Changes
1.0	2026-04-30	Project Team	Initial condensed release

Table 3.4: Document revision history.